

■ 서식 2 : 결과보고서 요약본

Project Based Learning 결과보고서 요약본

교과목명	국문	게임엔진응용			
	영문	Game Engine Application			
PBL 관련 능력단위 (능력단위코드)		PBL 관련 능력단위요소 (능력단위요소코드)			
게임엔진응용프로그래밍 (0803020527_16v3)		상용 게임엔진 활용하기 (0803020527_16v3.1)			
		상용 게임엔진 응용 프로그램 구현하기 (0803020527_16v3.2)			
학년 반	2학년 1반	조원	4명		
프로젝트 주제	병맛 혈압 마리오				
지도교수	이성현	산업체 참가여부	☐ 유 ☒ 무		
참여학생	임준섭, 은지원, 문선호, 문지훈				
작품 개요 (주제 선정 이유)	아이디어 평가기준	창의성	30		
		제작용이성	40		
		협업가능성	30		
	논의된 아이디어	아이디어 주제			
		1, 병맛 마리오			
		2, 총알 피하기			
	아이디어 평가 (팀원들 평가점수 평균값)	주제	창의성	제작용이성	협업가능성
병맛마리오		25	30	15	
총알피하기		15	25	20	
선정된 아이디어	병맛 마리오				
작품 구조도 (문제점 제시 및 개선방안)	1. 프로젝트 개요				
	1.1. 프로젝트 주제 밈(Meme) 캐릭터 '칠가이(Chill Guy)'를 활용하여, 사용자의 고정관념을 깨는 레벨 디자인과 특수 기믹(Gimmick)이 포함된 2D 플랫폼어 게임.				
	1.2. 개발 동기 및 필요성 평온한 표정의 캐릭터(칠가이)가 억울하고 부조리한 상황에서 죽는 모습을 통해 역설적인 재미를 주는 '부조리 게임(Troll Game)' 장르를 구현한다.				
	1.3. 개발 목표 - 캐릭터: '칠가이' 밈 캐릭터 구현 (좌우 이동, 점프). - 스테이지: 총 3개의 스테이지와 명확한 목표 오브젝트(화장실 관련) 배치.				

	• 시스템: 물리 엔진과 상호작용하는 함정 로직 및 엔딩 컷신 구조 설계. 2. 요구사항 분석 및 스테이지 설계 (Requirements Analysis) 본 프로젝트는 스테이지가 진행될수록 기믹이 심화되며, 각 스테이지는 '해소'를 위한 목표물을 향해 가는 여정이다. 2.1. 스테이지별 목표 및 상세 기믹 1 스테이지 • 목표(Goal): 맵 끝에 있는 '두루마리 휴지' 획득. - 기믹: • 투명 스파이크: 보이지 않는 바닥의 가시. • 투명 블록: 점프 경로를 방해하는 투명 벽. • 소멸 지형: 밟으면 떨어지는 바닥. 2 스테이지 • 목표(Goal): 맵 끝에 있는 '공중 화장실' 도착. - 기믹: • 1스테이지 기믹 계승. • 적(Enemy) 추가: 달으면 즉사하는 방해꾼 배치. • 함정 파이프: 비밀통로로 가는 길처럼 보이지만 사망으로 이끄는 가짜 파이프. 엔딩 (Ending): • 2스테이지 클리어 시, 캐릭터가 화장실에 들어간 후 평온을 찾는 스토리 컷신 재생 후 게임 종료
관련 이론	1. 플레이어 조작 시스템 <pre>참조 1개 public void mover() { // 1. 좌우 이동 처리 float moveInput = Input.GetAxisRaw("Horizontal"); // -1 (왼쪽), 1 (오른쪽), 0 (정지) rb.linearVelocity = new Vector2(moveInput * moveSpeed, rb.linearVelocity.y); if (moveInput < 0) { spriteRenderer.flipX = false; // 스프라이트를 X축 기준으로 뒤집기 } // moveInput 값이 0보다 크면 (오른쪽으로 이동) else if (moveInput > 0) { spriteRenderer.flipX = true; // 스프라이트를 원래 방향으로 } // 2. 점프 처리 // 윗방향키를 눌렀고, isGrounded가 true일 때만 점프 실행 if (Input.GetKeyDown(KeyCode.UpArrow) && isGrounded) { // 위쪽 방향으로 순간적인 힘(Impulse)을 가합니다. rb.AddForce(Vector2.up * jumpPower, ForceMode2D.Impulse); } }</pre>

```
[Header("이동 속도")]
public float moveSpeed = 5f; // 이동속도
```

```
[Header("점프 힘")]
public float jumpPower = 10f; // 점프 힘
```

(playermover.cs)

이동: 키보드 좌우 방향키를 통해 플레이어의 좌우 이동.

위 방향키를 통해 플레이어가 점프.

이동 속도 및 점프 힘에 값을 주어 속도감 부여.

```
public Transform player; // 플레이어의 Transform
public float xOffset = 0f; // X축으로 플레이어와 떨어질 거리
public float fixedYPosition; // 카메라가 고정될 Y축 높이

public float cameraMinX; // 카메라가 갈 수 있는 가장 왼쪽 X좌표
public float cameraMaxX; // 카메라가 갈 수 있는 가장 오른쪽 X좌표

// 만약 인스펙터에서 Y값을 지정하는 대신,
// 게임 시작 시의 카메라 Y 높이를 그대로 쓰고 싶다면
// private float startY;
// void Start()
// {
//     if (player == null) {
//         Debug.LogError("플레이어가 할당되지 않았습니다!");
//     }
//     startY = transform.position.y; // 현재 카메라의 Y 높이를 저장
// }

// LateUpdate는 플레이어의 움직임이 모두 끝난 후 실행되어 카메라 떨림을 방지합니다.
// Unity 메시지 | 참조 0개
void LateUpdate()
{
    if (player == null)
    {
        return; // 플레이어가 없으면 아무것도 안 함
    }

    // 1. 목표 X 위치 계산 (플레이어의 X위치 + X간격)
    float targetX = player.position.x + xOffset;

    // targetX 값을 cameraMinX와 cameraMaxX 사이로 강제로 고정시킵니다.
    targetX = Mathf.Clamp(targetX, cameraMinX, cameraMaxX);

    // 2. 목표 Y 위치는 고정값 사용
    float targetY = fixedYPosition;
    // (또는 Start에서 저장한 startY 값을 사용: float targetY = startY;)

    // 3. Z 위치는 현재 카메라의 Z 위치를 그대로 유지 (중요!)
    // (2D에서 카메라는 보통 Z축 -10에 있습니다)
    float targetZ = transform.position.z;

    // 4. 카메라 위치를 새로운 Vector3 값으로 설정
    transform.position = new Vector3(targetX, targetY, targetZ);
}
```

카메라가 플레이어를 추격하도록 만든 코드. ↑

(cameramove.cs)

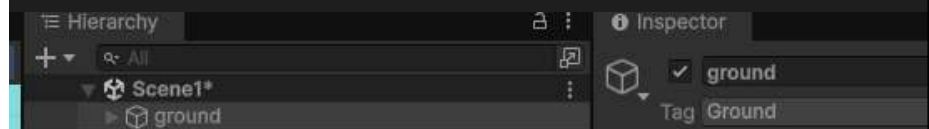
플레이어가 중앙에 위치하게 카메라 추적.



2. 땅 오브젝트 충돌 감지 및 처리

```
// 다른 Collider와 충돌을 시작할 때 호출됩니다.
// Unity 메시지 | 참조 0개
private void OnCollisionEnter2D(Collision2D collision)
{
    // 충돌한 오브젝트의 태그가 "Ground"이면
    if (collision.gameObject.CompareTag("Ground"))
    {
        isGrounded = true; // 땅에 닿았다고 표시
    }
}

// 다른 Collider와 충돌이 끝났을 때 호출됩니다.
// Unity 메시지 | 참조 0개
private void OnCollisionExit2D(Collision2D collision)
{
    // 떨어진 오브젝트의 태그가 "Ground"이면
    if (collision.gameObject.CompareTag("Ground"))
    {
        isGrounded = false; // 땅에서 떨어졌다고 표시
    }
}
```



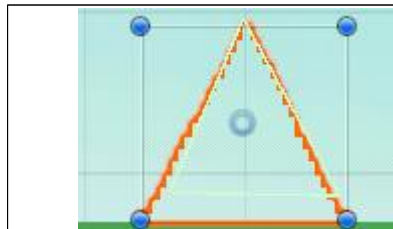
CompareTag를 활용하여 충돌 감지.
태그 'Ground'를 넣어 땅과 충돌하고 있음을 표시.
(playermover.cs)

3. 함정 시스템

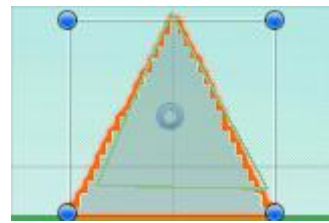
1) 투명 스파이크

```
private void OnTriggerEnter2D(Collider2D other)
{
    if (other.CompareTag("Player"))
    {
        Color visibleColor = spriteRenderer.color;
        visibleColor.a = 1f; // 알파 값을 1로 설정 (255에 해당)
        spriteRenderer.color = visibleColor;
    }
}
```

투명 스파이크, 플레이어와 충돌 시 컬러 값 변화로
스�파이크 위치 확인 및 플레이어 사망.
(spike.cs)



투명 스파이크 충돌 전

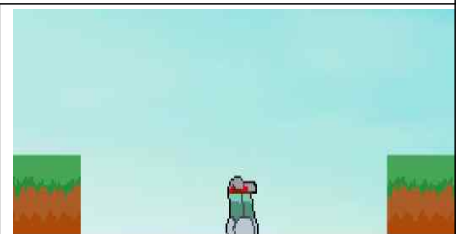


투명 스파이크 충돌 후

2) 사라지는 블록



기믹 블록 충돌 전



기믹 블록 충돌 후

```
private void OnTriggerEnter2D(Collider2D collision)
{
    if(tag == "gimic")
    {
        Destroy(gameObject);
    }
}
```

Tag를 주어서 플레이어와 충돌 시 오브젝트 삭제 및 플레이어 추락.
(gimic.cs)

3) 투명 블록

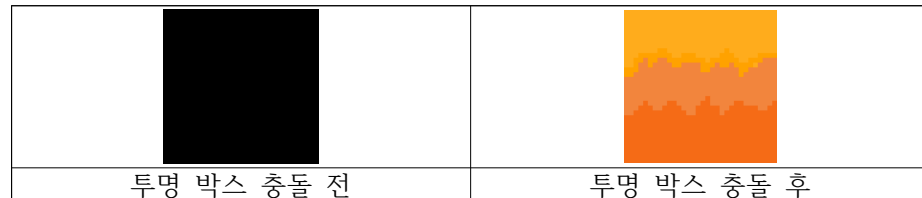
```
private void OnTriggerEnter2D(Collider2D collision)
{
    if(collision.CompareTag("Player"))
    {
        tumu.SetActive(true);
    }
}
```

(tumung.cs)

```
private void OnTriggerEnter2D(Collider2D collision)
{
    if(collision.CompareTag("tumung"))
    {
        rb.linearVelocity = new Vector2(rb.linearVelocity.x, Yvlaue);
    }
}
```

투명 블록과 충돌 시 비활성화인 투명 블록 활성화.
'tumung' 태그와 충돌 시 Y축 값 변경으로 플레이어 추락.

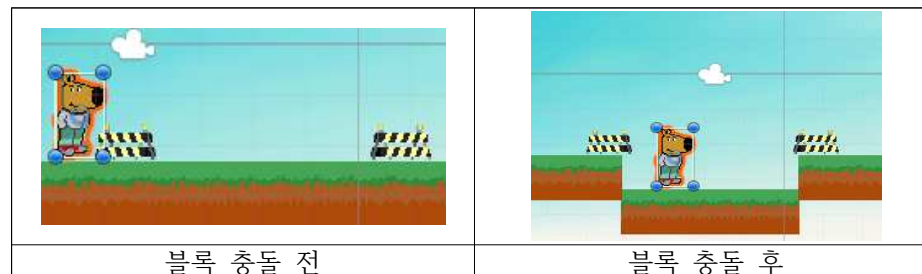
(deing.cs)



4) 떨어지는 블록

```
private void OnTriggerEnter2D(Collider2D collision)
{
    if(collision.CompareTag("Player"))
    {
        rb.gravityScale = 1;
    }
}
```

플레이어와 충돌 시 중력 값으로 낙하 및 플레이어 추락.
(groundgimic.cs)



5) 죽는 파이프

```
public class p1 : MonoBehaviour
{
    public KeyCode enterKey = KeyCode.UpArrow; // 윗방향키 설정
    public Vector3 enterDirection = Vector3.down; // 파이프 안으로 들어가는 방향 (보통 아래)
    public float enterDuration = 1.0f; // 들어가는 데 걸리는 시간

    private bool isStandingOnPipe = false; // 파이프 위 확인
    private GameObject player;
    private bool isEntering = false; // 파이프에 들어가는 중인가?

    // 플레이어가 파이프 입구(트리거)에 닿았을 때
    // Unity 메시지 참조 0개
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            isStandingOnPipe = true;
            player = collision.gameObject;
        }
    }

    // 플레이어가 파이프 입구에서 나갔을 때
    // Unity 메시지 참조 0개
    private void OnTriggerExit2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            isStandingOnPipe = false;
            player = null;
        }
    }

    private void Update()
    {
        // 파이프 위에 있고, 아직 들어가는 중이 아니며, 지정된 키를 눌렀을 때
        if (isStandingOnPipe && !isEntering && Input.GetKeyDown(KeyCode.DownArrow))
        {
            if (player != null)
            {
                StartCoroutine(EnterPipeRoutine(player));
            }
        }
    }
}

// 참조 1개
IEnumerator EnterPipeRoutine(GameObject target)
{
    playermover playerScript = target.GetComponent<playermover>();
    isEntering = true;

    // 주의: 사용하시는 플레이어 이동 스크립트 비활성화.
    if (playerScript != null)
    {
        playerScript.enabled = false;
    }

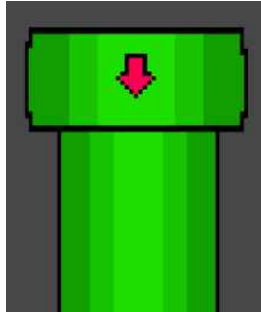
    Rigidbody2D rb = target.GetComponent<Rigidbody2D>();
    if (rb != null) rb.simulated = false; // 물리 연산 끄기 (중력 무시 등)

    // 2. 플레이어 위치를 파이프 중앙으로 살짝 정렬 (선택 사항)
    Vector3 startPos = target.transform.position;
    Vector3 targetPos = startPos + enterDirection; // 현재 위치에서 지정된 방향으로 이동

    // 3. 부드럽게 안으로 들어가는 애니메이션
    float elapsed = 0f;
    while (elapsed < enterDuration)
    {
        target.transform.position = Vector3.Lerp(startPos, targetPos, elapsed / enterDuration);
        elapsed += Time.deltaTime;
        yield return null;
    }

    // 4. 애니메이션 끝난 죽음
    playerScript.Die();
}
```

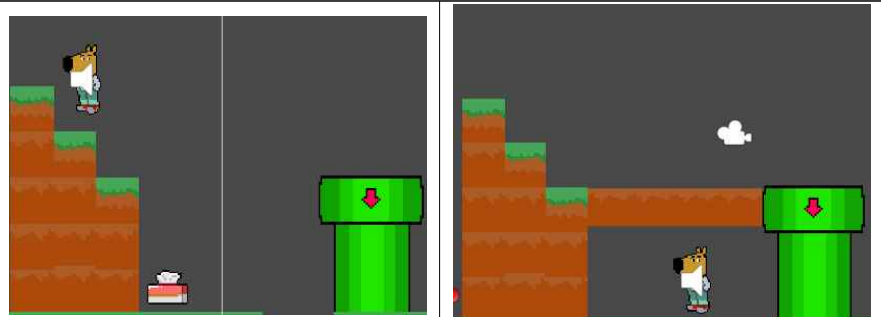
파이프에 들어가면 죽는 기믹. 화살표로 표시해서 들어가고 싶은 느낌을 줌.
Vector 값을 넣어 중앙, 속도 조절해서 파이프에 들어가는 모션.
(pi.cs)



6) 플레이어 가두기

아이템인 척 하는 휴지를 먹으러 이동할 시 투명 블록으로 막혀 진행 못하게 막는 기믹. 유일한 탈출법은 플레이어 사망.

투명 블록 기믹은 3) 투명 블록과 동일



가두기 전

가두기 후

4. 적 (enemy)

```
void Update()
{
    if (isturn)
    {
        transform.Translate(Vector2.right * 1.5f * Time.deltaTime);
        spriteRenderer.flipX = true;
    }
    else {
        spriteRenderer.flipX = false;
        transform.Translate(Vector2.left * 1.5f * Time.deltaTime);
    }
}
```

적 이동 로직 (enemymove.cs)

```
private void OnCollisionEnter2D(Collision2D collision)
{
    //플레이어 스크립트를 불러온다
    playermove playerScript = collision.gameObject.GetComponent<playermove>();
    if (collision.gameObject.CompareTag("endline"))
    {
        isturn = true;
    }
    if (collision.gameObject.CompareTag("Ground2"))
    {
        isturn = false;
    }
    if (collision.gameObject.CompareTag("Player"))//플레이어에게 닿으면 몬스터 죽음
    {
        playerScript.Die();
    }
}
```

1) 벽에 부딪히면 이동 방향 반대로 뒤집기

2) 플레이어와 충돌 시 플레이어 사망

(enemymove.cs)

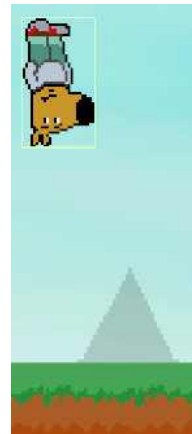
```
private void OnTriggerEnter2D(Collider2D collision)
{
    //플레이어가 머리를 밟으면 부모객체에게 있는 enemymove에 enemyDie()를 쓴다.
    if (collision.gameObject.CompareTag("Player"))
    {
        enemymove.enemyDie();
    }
}

public void enemyDie()
{
    Destroy(gameObject);
}
```

플레이어가 적(enemy)의 머리를 밟으면 적(enemy) 사망

(enemyhede.cs)

5. 사망 액션 효과



```
private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.gameObject.CompareTag("spike"))
    {
        Die();
    }
}

참조 2개
public void Die()
{
    if (isDead) return;
    isDead = true;
    rb.linearVelocity = Vector2.zero; // 현재 속도를 0으로 초기화
    rb.AddForce(Vector2.up * 10, ForceMode2D.Impulse);

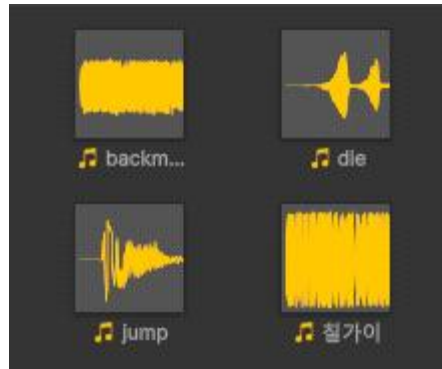
    // 2. 스프라이트를 수직으로 뒤집습니다.
    spriteRenderer.flipY = true;
    cl.isTrigger = true;
}
```

스파이크 충돌

혹은 Y축 값 = 10 일 시 플레이어 뒤집히며 떨어짐 및

사망 액션.
(playermover.cs)

6. 소리 효과



backmusic : 플레이 스테이지 브금 (투스리스 댄스 밌 브금)

die : 플레이어 사망 시 사운드

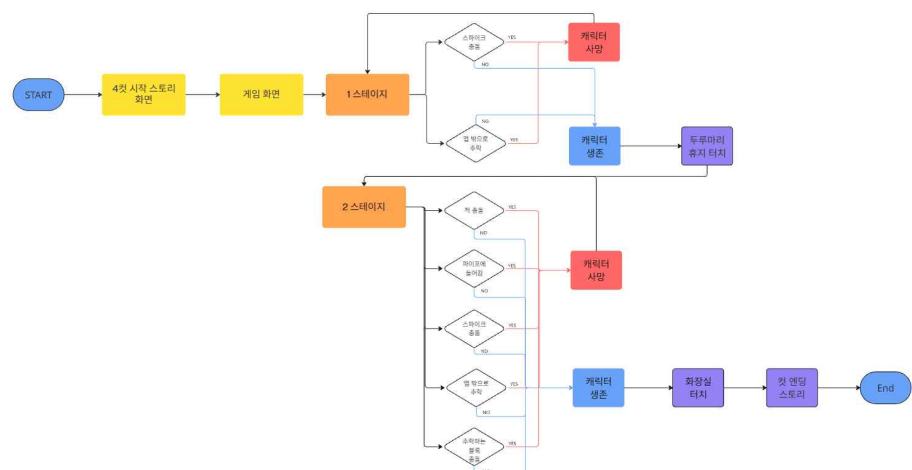
jump : 플레이어 상 방향카 누를 시 (점프 할 시) 사운드

철가이 : 처음 시작, 메뉴 화면 사운드 (철가이 밌 브금)

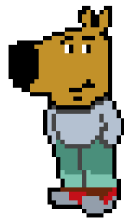
7. 씬 전환

```
private void OnTriggerEnter2D(Collider2D collision)
{
    if(collision.CompareTag("Player"))
    {
        gameObject.SetActive(false);
        SceneManager.LoadScene("Scene2");
    }
}
```

(item1.cs)



결과물 제작
(문제점 개선사항)



플레이어 '칠 가이'



'칠 가이' 사망 모션



땅 오브젝트



공중 박스 오브젝트



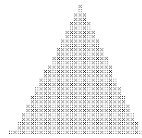
각티슈
(가짜 아이템)



두루마리 휴지
(2 스테이지 목표)



공중 화장실 오브젝트
(엔딩)



투명 스파이크
(기믹 함정)



투명 박스 (기믹 함정)



적(enemy)

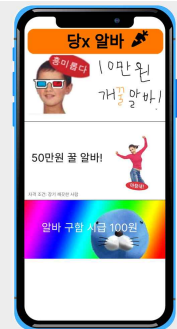


함정 파이프

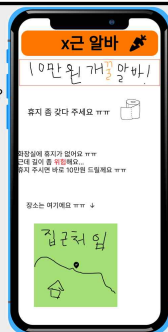
돈이 없어....
돈이 필요하군...



오.. 10만원
꿀알바라니...



뭐야 휴지만 주면 10만원?
정말 Chill 하군...
심지어 집 근처야...



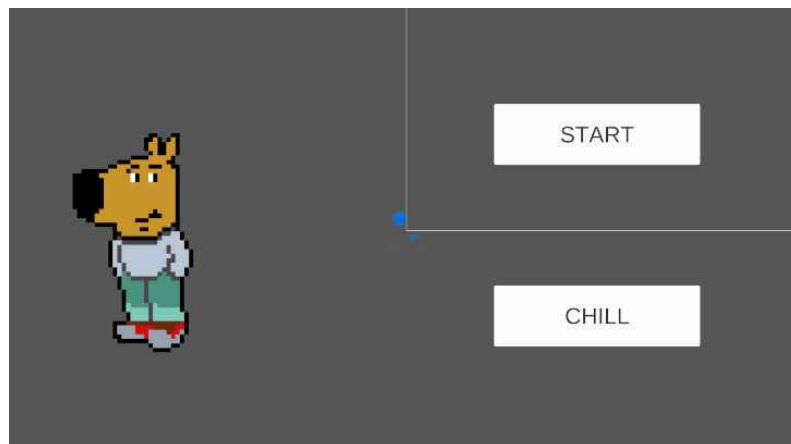
당장 간다
기다려라
나의 10만원



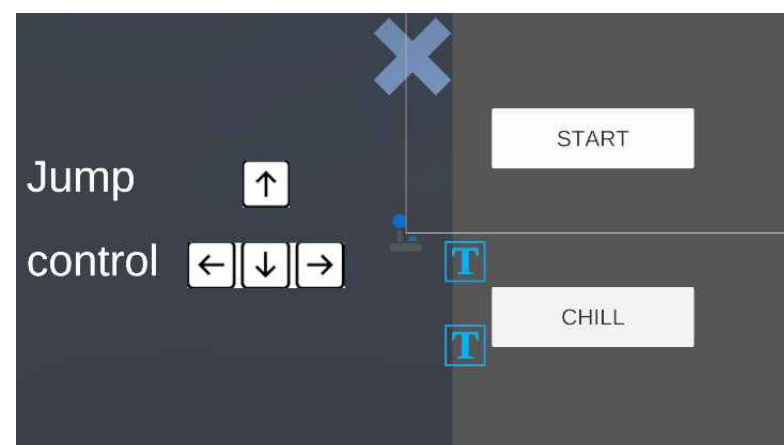


2 스테이지 클리어 후 스토리 만화(엔딩)

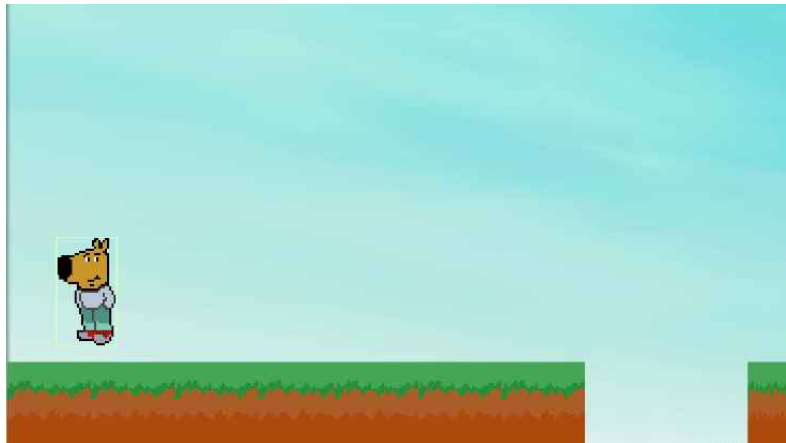
- 사람이 있는 엔딩은 맞지 않다고 판단하여 마지막 재치있게 엔딩 작성



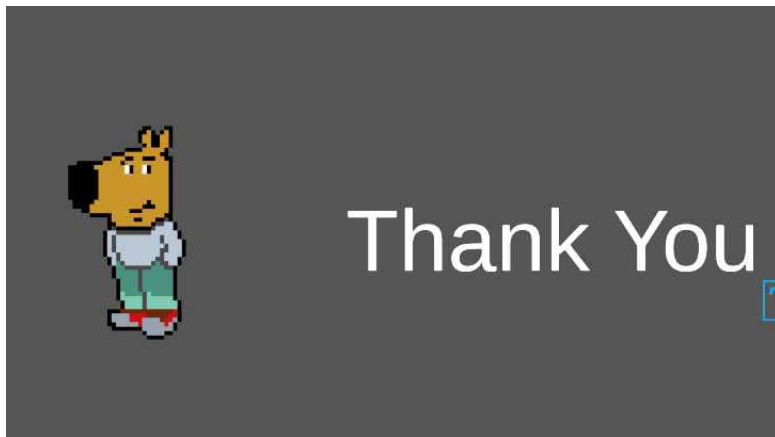
시작 및 메뉴 화면



Chill 클릭 시 나오는 설명 화면



게임 시작 화면



엔딩 화면

기대 효과 및 활용방안

1. 기술적 및 교육적 기대 효과
 - **고급 물리 로직 습득:** 일반적인 충돌(Collision)과 이벤트 트리거(Trigger)의 차이를 명확히 이해하고, 보이지 않는 오브젝트와의 상호작용을 구현하며 물리 엔진 활용 능력을 극대화함.
 - **레벨 디자인 역량 강화:** 플레이어의 심리를 예측하고 역이용하는 설계를 통해, 단순 구현을 넘어선 '경험(Experience)' 중심의 기획력을 배양함.
 - **문제 해결 능력 향상:** 게임의 불합리함을 '재미'로 승화시키는 과정에서 발생하는 밸런스 문제를 기술적으로 보완하며 실무적인 디버깅 능력을 기름.
2. 콘텐츠 활용 방안
 - **포트폴리오 활용:** 기획, 리소스(밌) 활용, 로직 구현, 엔딩까지의 전체 개발 사이클을 완수한 결과물로서, 게임 개발 역량을 입증하는 포트폴리오로 활용 가능.

2025 년 11 월 24 일

지도교수 이 성 현 (인)